



EDUCACIÓN
SECRETARÍA DE EDUCACIÓN PÚBLICA



TECNOLÓGICO
NACIONAL DE MÉXICO



Instituto Tecnológico de San Juan del Río



Tópicos de ciberseguridad

Actividad: Parchar un ejecutable

Ingeniería en sistemas Computacionales

P R E S E N T A:

Angelica Enriquez Yañez 22590325

Daniela Dorantes Callejas 22590323

Julieta Becerril Ruiz 22590320

PERIODO [Agosto - Diciembre]



Objetivo

Identificar la función principal `main ()` del programa, localizar las instrucciones condicionales responsables de ocultar o bloquear determinados elementos de la interfaz, y modificar dichas instrucciones mediante técnicas de parchado para restablecer el flujo de ejecución esperado. Adicionalmente, se buscó comprender la diferencia entre los datos persistentes del programa (almacenados en una base de datos local) y el código del ejecutable que define su lógica de funcionamiento.

Herramientas utilizadas

- **x32dbg**: depurador a nivel de ensamblador que permite la ejecución paso a paso del programa, la inspección de registros del procesador y la modificación de instrucciones en tiempo de ejecución.
- **Ghidra**: plataforma de análisis de código fuente y desensamblado desarrollada por la NSA, que facilita la búsqueda de cadenas de texto, la identificación de referencias y la comprensión general de la estructura del binario.

Desarrollo

En primer lugar, se cargó el archivo `phackeame.exe` en Ghidra y se permitió que el analizador automático procesara el binario completo. Una vez finalizado, se accedió al Symbol Tree y se localizó la función `entry`, que es el verdadero punto de entrada del ejecutable tal como lo define el sistema operativo.

Al examinar el decompiler de `entry`, se encontró el siguiente código:



```
00402cee 00      ??      00h
00402cef 00      ??      00h

*****
*                               *
*                               *
*****

undefined entry()
<UNASSIGNED> <RETURN>

entry
XREF[2]:  Entry Point(*), 004000a8
00402cf0 c6 05 c0 MOV     byte ptr [DAT_0055f4c0],0x0
f4 55 00 00
00402cf7 b9 00 90 MOV     ECX,PTR_tls_callback_0_005f9000 = 00402c70
5f 00
00402cfc ba 04 90 MOV     EDX,DAT_005f9004
5f 00
.....
```

```
1 void entry(void)
2 {
3
4 {
5     DAT_0055f4c0 = 0;
6     FUN_00402c50();
7     FUN_00402c60();
8     FUN_004146e0();
9     return;
10 }
11 }
```

Las dos primeras funciones resultaron triviales al inspeccionarlas: una retorna inmediatamente y la otra realiza una asignación simple de variable global. Toda la lógica de inicialización recayó en `FUN_004146e0`.

[illegible]

Al abrir el Listing (vista ensamblador) de FUN_004146e0, se observó que la función realiza la inicialización estándar del CRT (C Runtime) de MSVC: configura el stack de excepciones SEH, guarda registros de segmento y establece el manejador de errores. La instrucción clave que lleva al main real del programa fue encontrada en la dirección 004146c4:

Address	Disassembly	Comment	Flow
004146c4	CALL	dword ptr [DAT_005d231c]	
004146ca	PUSH	dword ptr [DAT_005d1010]	
004146d0	CALL	KERNEL32.DLL::ExitProcess	
-- Flow Override: CALL_RETURN (CALL_TERMINATOR)			
004146d5	??	89h	
004146d6	??	ECh	
004146d7	??	5Dh]	
004146d8	??	C3h	

una llamada a través de un puntero almacenado en .bss es exactamente como el CRT de MSVC entrega el control al main. Ghidra mostraba = ?? en la dirección 005cd0c4 porque dicho puntero se resuelve en tiempo de ejecución: la sección .bss no tiene valor inicial en el archivo, sólo se llena cuando el programa arranca.

Address	Disassembly	Comment
005cd0c4	06 ?? 06h	DAI_005cd0c4 XREF[1]: 005cd110 (*)
005cd0c5	49 6e 73 ds "Insert"	
005cd0c6	65 72 74 00	
005cd0cc	04 ?? 04h	DAI_005cd0cc XREF[1]: 005cd118 (*)
005cd0cd	4d ?? 4Dh M	
005cd0ce	6f ?? 6Fh o	



Sin embargo, durante esta etapa se presentó la dificultad de que el punto de entrada no correspondía directamente con la función *main*, por lo que resultó complejo identificar el flujo principal del código desde el depurador.

Ante esta situación, se recurrió a x32dbg:

Se abrió phackeame.exe en x32dbg. Con Ctrl+G se navegó a la dirección 00414721. Se colocó un breakpoint con F2 (la línea se marcó en rojo).

EIP	Address	Disassembly	Comment
00414721	FF15 14235D00	call dword ptr ds:[5D2314]	
00414727	5D	pop ebp	
00414728	E8 C3FEFFFF	call phackeame.4145F0	
0041472D	89EC	mov esp,ebp	
0041472F	5D	pop ebp	
00414730	C3	ret	
00414731	0000	add byte ptr ds:[eax],al	
00414733	0000	add byte ptr ds:[eax],al	
00414735	0000	add byte ptr ds:[eax],al	
00414738	0000	add byte ptr ds:[eax],al	
0041473D	0000	add byte ptr ds:[eax],al	
0041473F	0055 89	add byte ptr ss:[ebp-77],d1	
00414742	E5 8D	inc eax,8D	
00414744	64:24 E4	and al,E4	
00414747	53	push ebx	
00414748	C645 EC 00	mov byte ptr ss:[ebp-14],0	
0041474C	6A 00	push 0	
0041474E	6A 10	push 10	
00414750	8D55 F0	lea edx,dword ptr ss:[ebp-10]	
00414753	52	push edx	
00414754	50	push eax	
00414755	E8 66C9FEFF	call <JMP.&GetCurrentProcess>	
0041475A	50	push eax	
0041475B	E8 70C9FEFF	call <JMP.&ReadProcessMemory>	
00414760	85C0	test eax,eax	
00414762	0F84 BC000000	je phackeame.414824	
00414768	31C9	xor ecx,ecx	
0041476A	B3 01	mov bl,1	
0041476C	EB 46000000	jmp phackeame.414817	

Se ejecutó el programa con F9.x32dbg pausó en esa instrucción exacta. En ese momento el panel de disassembly mostró la dirección destino ya resuelta.Se presionó F7 (step into) para entrar a la función apuntada.El EIP saltó a la dirección 00402D20, que es el main real del programa.

EIP	00402D20	55	push ebp
	00402D21	89E5	mov ebp,esp
	00402D23	E8 F8C70000	call phackeame.40F520
	00402D28	C605 30015600 01	mov byte ptr ds:[560130],1
	00402D2F	A1 00015600	mov eax,dword ptr ds:[560100]
	00402D34	C680 58010000 01	mov byte ptr ds:[eax+158],1
	00402D38	A1 00015600	mov eax,dword ptr ds:[560100]
	00402D40	8B15 00015600	mov edx,dword ptr ds:[560100]
	00402D46	8B12	mov edx,dword ptr ds:[edx]
	00402D48	FF92 F4000000	call dword ptr ds:[edx+F4]
	00402D4E	B9 20515F00	mov ecx,phackeame.5F5120
	00402D53	A1 00015600	mov eax,dword ptr ds:[560100]
	00402D58	BA 900D5800	mov edx,phackeame.580D90
	00402D5D	E8 CE4C0200	call phackeame.427A30
	00402D62	A1 00015600	mov eax,dword ptr ds:[560100]
	00402D67	E8 84380200	call phackeame.4265F0
	00402D6C	E8 DFCA0000	call phackeame.40F850
	00402D71	89EC	mov esp,ebp
	00402D73	5D	pop ebp
	00402D74	C3	ret
	00402D75	0000	add byte ptr ds:[eax],al
	00402D77	0000	add byte ptr ds:[eax],al
	00402D79	0000	add byte ptr ds:[eax],al
	00402D7B	0000	add byte ptr ds:[eax],al
	00402D7D	0000	add byte ptr ds:[eax],al
	00402D7F	0064A1 04	add byte ptr ds:[ecx+4],ah
	00402D83	0000	add byte ptr ds:[eax],al
	00402D85	00C3	add bl,al
	00402D87	0000	add byte ptr ds:[eax],al
	00402D89	0000	add byte ptr ds:[eax],al
	00402D8B	0000	add byte ptr ds:[eax],al

Para encontrar exactamente qué función del programa es responsable de deshabilitar el botón "Presioname!", se utilizó la técnica de breakpoint en API de Windows. Cuando un programa Win32 oculta o deshabilita un control de ventana, necesariamente llama a la función del sistema EnableWindow de la librería user32.dll. Colocando un breakpoint en esa función se puede atrapar el momento exacto en que el programa manipula el botón.

Procedimiento en Ghidra:

Se abrió la pestaña Symbols y se buscó user32 -EnableWindow.

Name	Location	Type	Namespace	Source	Reference Count	Offset Ref Count
EnableWindow	External[001fbc16]	External Function	USER32.DLL	Imported	3	0
EnableWindow	00401d70	Thunk Function	USER32.DLL	Default	3	0

Location	Label	Code Unit	Context
004e0af7		CALL USER32.DLL::EnableW...	UNCONDITIONAL_CALL
004e0ca8		CALL USER32.DLL::EnableW...	UNCONDITIONAL_CALL
00557826		CALL USER32.DLL::EnableW...	UNCONDITIONAL_CALL

Con la dirección 004e0b50 identificada, se navegó a esa función en Ghidra. El decompiler la mostró como:

```

004e0b5d 8b 82 14    MOV     EAX,dword ptr [EDX + 0x214]
02 00 00
004e0b63 89 45 ec    MOV     dword ptr [EBP + local_18],EAX
004e0b66 85 c0       TEST    EAX,EAX
004e0b68 74 13       JZ      LAB_004e0b7d
004e0b6a 8b 45 ec    MOV     EAX,dword ptr [EBP + local_18]
004e0b6d 8b 00       MOV     EAX,dword ptr [EAX]
004e0b6f ba 28 ce    MOV     EDX,DAI_0057ce28
57 00
004e0b74 e8 27 cd    CALL    FUN_0040d8a0
f2 ff
004e0b79 84 c0       TEST    AL,AL
004e0b7b 74 06       JZ      LAB_004e0b83

LAB_004e0b7d
004e0b7d c6 45 f4 01 MOV     byte ptr [EBP + local_10],0x1
004e0b81 eb 04       JMP     LAB_004e0b87

LAB_004e0b83
004e0b83 c6 45 f4 00 MOV     byte ptr [EBP + local_10],0x0

LAB_004e0b87
004e0b87 80 7d f4 00 CMP     byte ptr [EBP + local_10],0x0

```

```

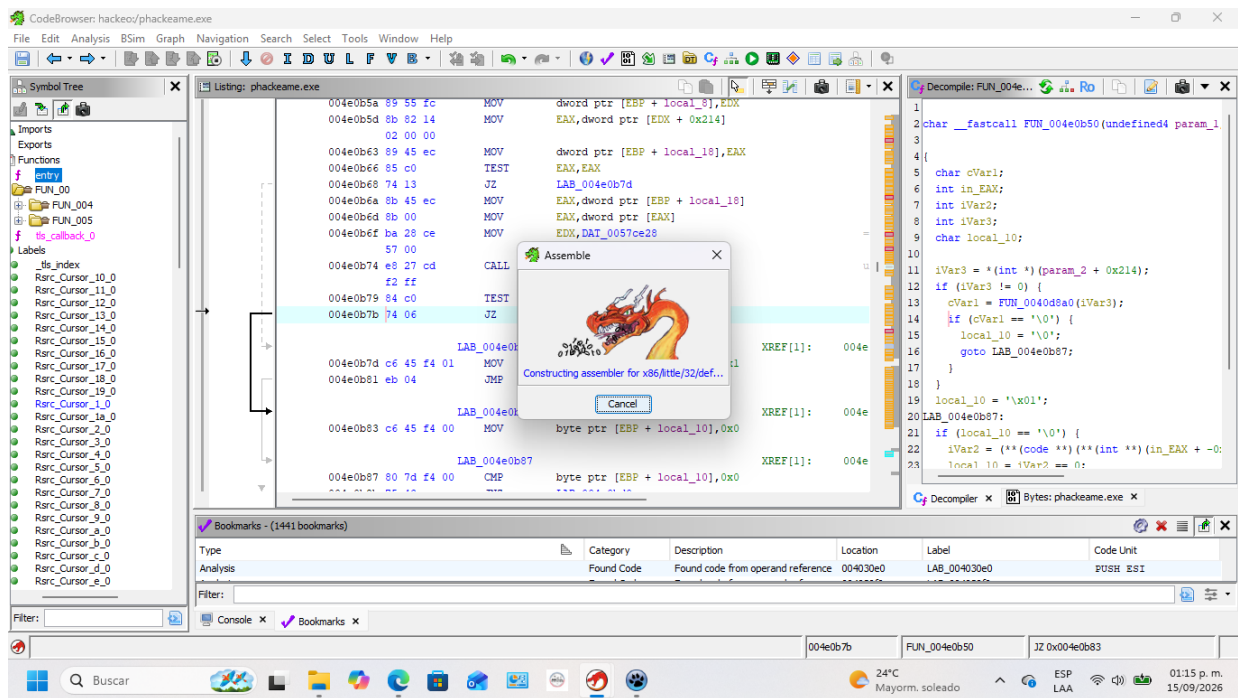
2 char __fastcall FUN_004e0b50(undefined4 param_1)
3
4 {
5     char cVar1;
6     int in_EAX;
7     int iVar2;
8     int iVar3;
9     char local_10;
10
11     iVar3 = *(int *) (param_2 + 0x214);
12     if (iVar3 != 0) {
13         cVar1 = FUN_0040d8a0(iVar3);
14         if (cVar1 == '\0') {
15             local_10 = '\0';
16             goto LAB_004e0b87;
17         }
18     }
19     local_10 = '\x01';
20 LAB_004e0b87:
21     if (local_10 == '\0') {
22         iVar2 = (**(code **)) (**(int **)) (in_EAX + -0:
23         local_10 = iVar2 == 0;

```

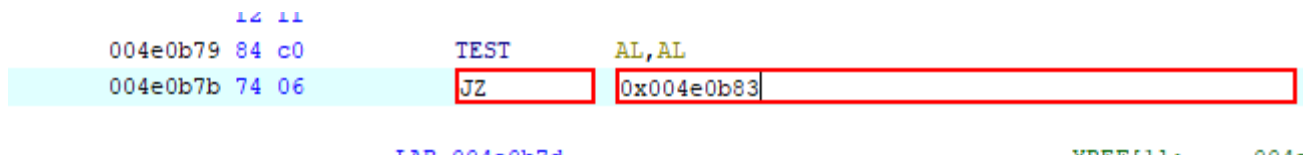
La función verifica una condición (relacionada al contador de intentos a través de FUN_0040d8a0). Si esa verificación falla, local_10 queda en '\0' (cero), lo cual provoca que más adelante se llame a EnableWindow(hButton, FALSE), ocultando y deshabilitando el botón "Presioname!". El flujo original era:

- Si la verificación de intentos falla → JZ salta a 004e0b83 → pone 0 → deshabilita botón
- Si la verificación pasa → continúa en 004e0b7d → pone 1 → habilita botón

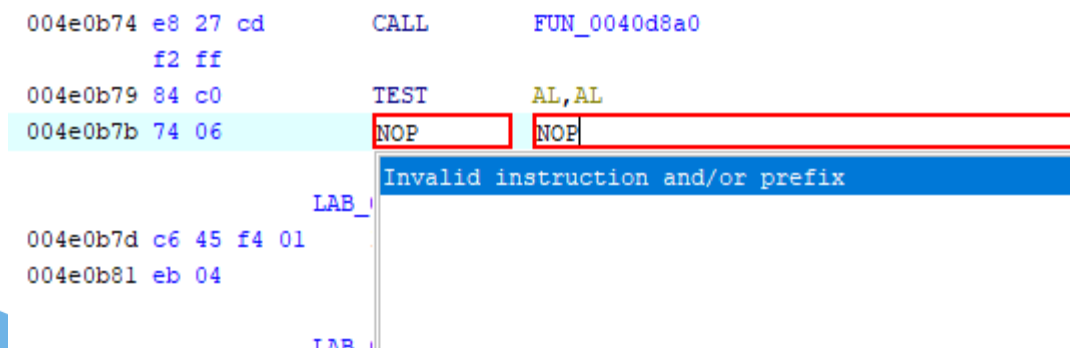
La instrucción JZ en 004e0b7b con opcode 74 06 (2 bytes) era el responsable por lo cual se procedió a aplicar el parche directamente en Ghidra mediante el Patch Instruction:



Se seleccionó la instrucción JZ LAB_004e0b83 en 004e0b7b

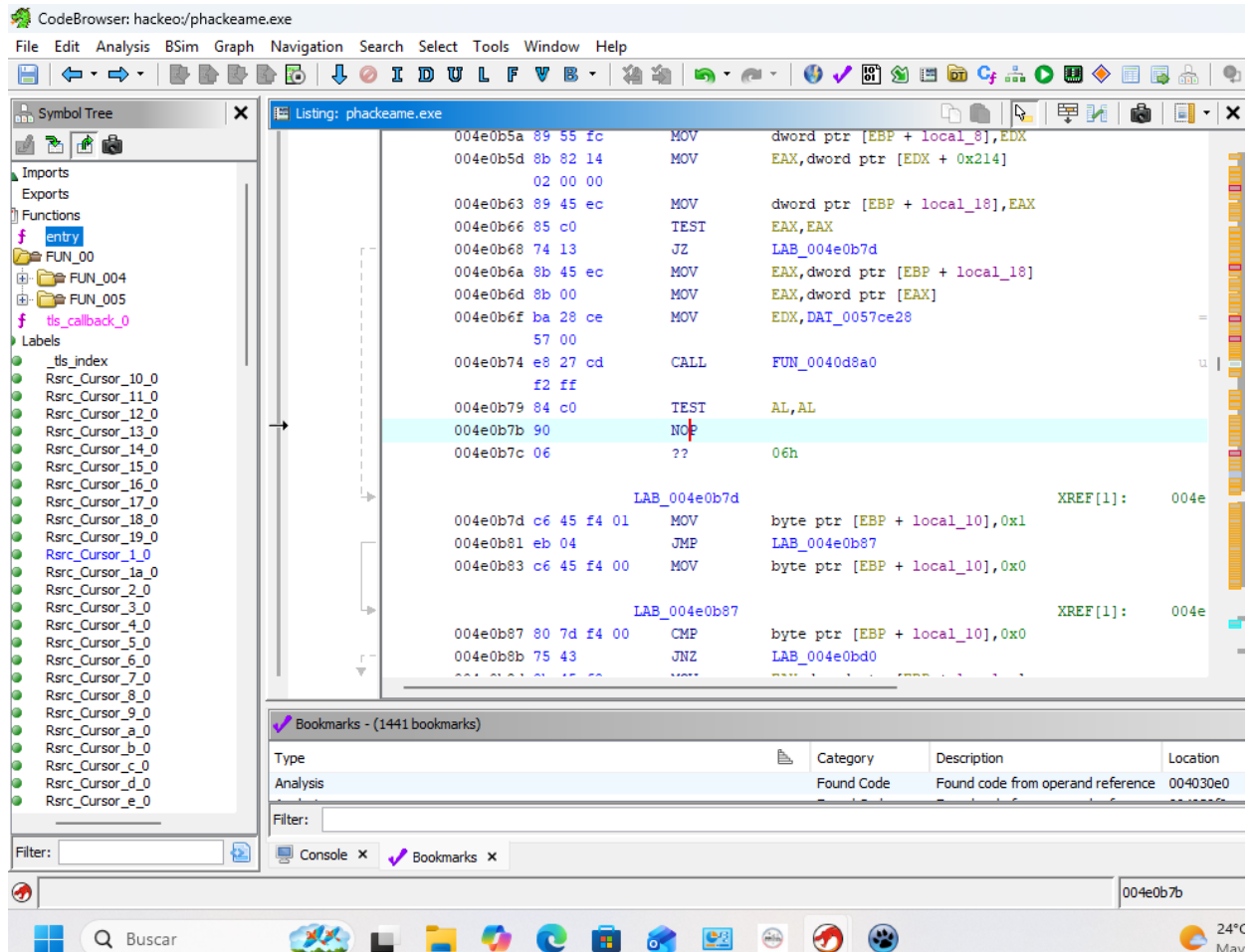


Se reemplazaron sus 2 bytes (74 06) por dos instrucciones NOP (90 90)



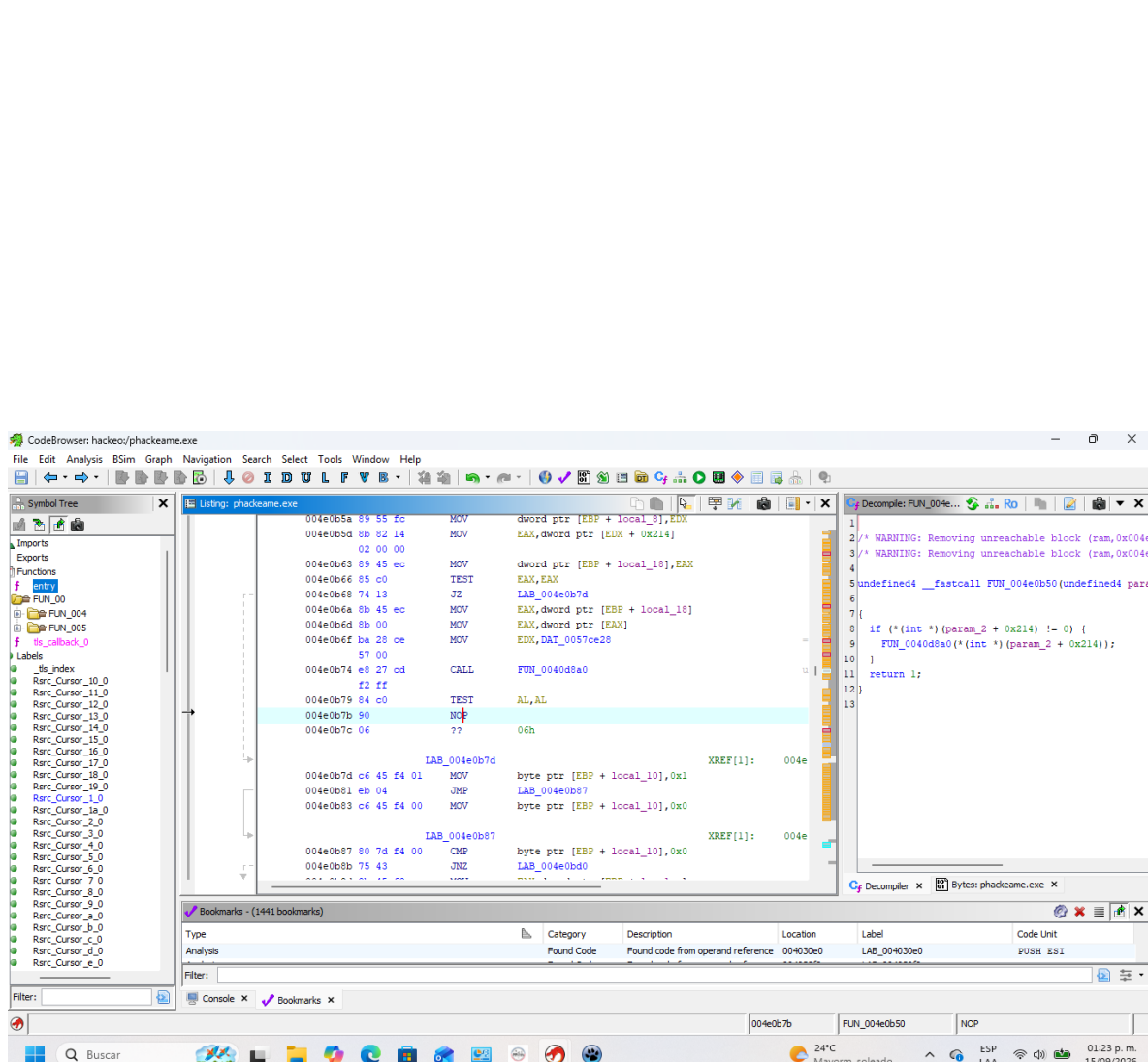
Con este cambio la ejecución nunca toma el camino que deshabilita el botón. El salto condicional fue reemplazado por dos operaciones NOP (No Operation no hacen nada), por

lo que el flujo cae directamente a 004e0b7d, que pone el valor 1 (habilitar), sin importar el resultado de la verificación de intentos.

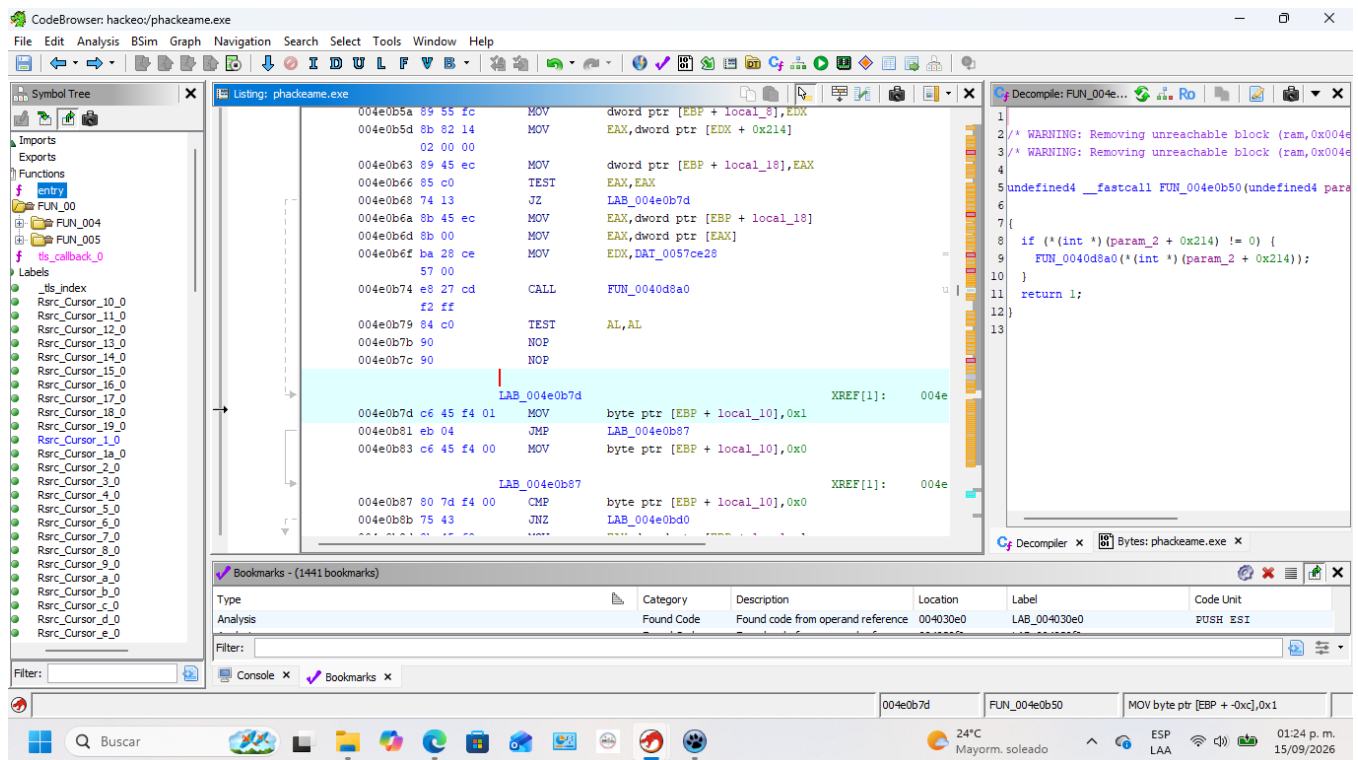


ASI quedo la función:

```
undefined4 __fastcall FUN_004e0b50(undefined4 para
{
    if (*(int *) (param_2 + 0x214) != 0) {
        FUN_0040d8a0(*(int *) (param_2 + 0x214));
    }
    return 1;
}
```

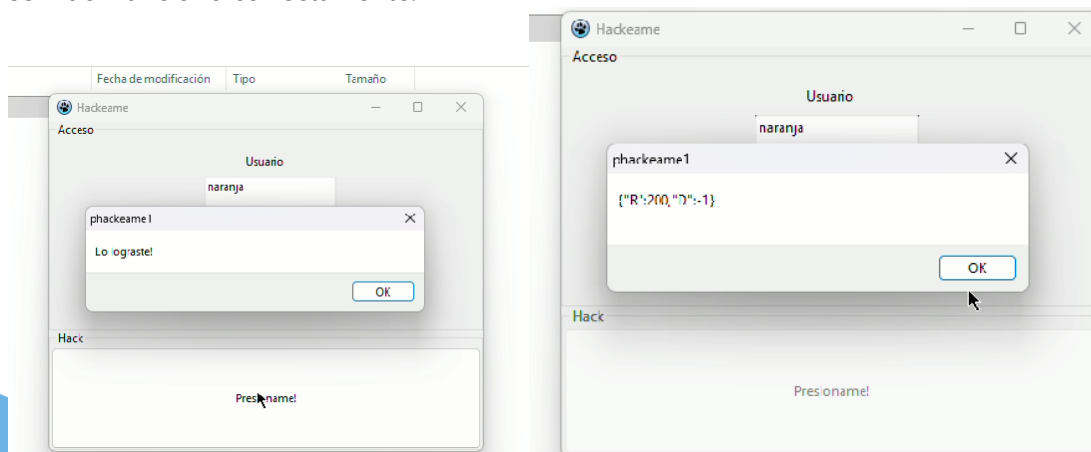
El 74 se convirtió en 90 (primer NOP), pero el 06 (el offset del salto) quedó ahí como byte sin interpretar (??). Eso no es un NOP: es basura. Si se deja así, cuando el procesador llegue a 004e0b7c, intentará ejecutar 06 como instrucción (en x86, 06 es PUSH ES, una instrucción válida pero que romperá el flujo). Hay que convertirlo también en 90



Con el parche aplicado se ejecutó el programa. El botón "Presioname!" permaneció visible en todo momento. Al ingresar las credenciales correctas en los campos de **Usuario** y **Contraseña** y presionar el botón "Acceso", seguido de "Presioname!", el programa mostró el siguiente mensaje:

"Lo lograste!"

Confirmando que el parche fue exitoso: la restricción visual fue eliminada y la conexión al servidor funcionó correctamente.



EL RESULTADO ESTA EN EL VIDEO ;)